

Design Notes & Key Decisions

This document captures the reasoning behind the key architectural and operational decisions in this project. Its purpose is to make the **thinking process** explicit — the *why* behind each choice, the trade-offs considered, and the limitations acknowledged honestly.

1. Environment & Platform

Decision: Ubuntu 24.04 LTS on AWS EC2 t3.xlarge (4 vCPU, 16 GB RAM, 40 GB gp3), ap-south-1 (Mumbai).

Reasoning:

- The assignment specified "Ubuntu 22.04 or similar." Ubuntu 22.04 was not available in the AWS Mumbai Quick Start AMIs at launch time, so 24.04 LTS was used — an equivalent LTS release satisfying "or similar." (*Documented as a deviation.*)
- RAM was raised from the 4 GB minimum to 16 GB because the observability stack (Prometheus + Grafana + Loki + node exporters) is memory-hungry on a single node. The assignment explicitly allows "adjust if running heavier services."

Trade-off: A smaller instance (t3.large, 8 GB) would be cheaper, but risked OOM-kills once monitoring was added. Chose reliability over marginal cost.

Cost control: The VM is stopped when not in active use (billing pauses; only EBS storage is charged). An

Elastic IP keeps the public address stable across stop/start cycles, so URLs and CI/CD targets don't change.

2. Application Design

Decision: A single Python Flask "Notes API" microservice backed by PostgreSQL, serving its own HTML frontend.

Reasoning:

- The evaluation is about SRE skills, not application complexity. The assignment says "a simple microservice app... can use a sample app or a small app."
- The app is deliberately a *vehicle* for demonstrating deploy/observe/break/recover. It is minimal but includes every hook needed: `/health` (probes), `/metrics` (Prometheus), a PostgreSQL dependency (enables the DB-failure scenario), and `/api/stress` (enables the CPU-exhaustion scenario).

Trade-off considered: A React frontend + multiple services was considered but rejected — it would consume time better spent on observability and incident response, which is what the evaluation actually weights.

Bugs found & fixed during development (real operational learning):

1. `init_db()` originally ran only under `if __name__ == "__main__"`, which gunicorn does not execute (it imports the module). Moved to import scope so the table is created under gunicorn. Surfaced by reading pod logs.

2. `metrics.counter()` from `prometheus-flask-exporter` is a decorator, not a counter object — switched to `prometheus_client.Counter`. Surfaced from an error log.

3. Containerization

Decision: `python:3.12-slim` base, non-root user, gunicorn as the WSGI server.

Reasoning:

- Slim base reduces image size and attack surface.
 - Running as a non-root `appuser` limits blast radius if the container is compromised.
 - gunicorn (not Flask's dev server) is production-appropriate; the dev server is single-threaded and explicitly not for production.
 - Images are tagged with the git commit SHA (immutable), not just `:latest`. This solved a real caching problem where k3s reused a stale `:latest` image.
-

4. Helm Chart

Decision: A single chart deploying app + Postgres + all supporting resources.

Reasoning:

- Config split into ConfigMap (non-secret: DB host/name/user/port) and Secret (DB password) — no secrets in Git.
 - Resource requests/limits prevent a single pod from starving the node (also the safety ceiling for the High-CPU scenario).
 - Liveness/readiness probes on `/health` enable self-healing and traffic gating.
 - 2 app replicas for high availability (see §6).
-

5. CI/CD

Decision: GitHub Actions — build, Trivy scan, push to ghcr.io, deploy to k3s over SSH.

Reasoning:

- ghcr.io chosen over Docker Hub: free, authenticates with the built-in `GITHUB_TOKEN` (no extra secret), no pull rate limits.
 - Images tagged with the git commit SHA for reproducible, auditable deploys.
 - Trivy scans every image for CRITICAL/HIGH CVEs (report-only for a PoC; production would fail the build on CRITICAL).
 - **Deploy runs as the non-root `ubuntu` user with a scoped, user-owned kubeconfig — NOT with `root/sudo`.** Least privilege: if CI credentials leaked, the blast radius excludes host root. Chosen deliberately over the simpler `sudo` approach.
-

6. High Availability vs Self-Healing (a deliberate distinction)

Decision: Run 2 replicas of the app.

Reasoning:

- With 2 replicas, killing one pod causes **zero downtime** (HA) — the other serves traffic while Kubernetes recreates the dead pod (**self-healing**).
- With 1 replica, killing the pod causes a brief outage, then self-healing recovery.
- The demo shows both: HA (kill 1 of 2, app stays up), then scale to 1 and show self-healing (brief outage + auto-recovery), making the *difference* explicit.

Key insight (validated by the Pod Crash Loop incident): a bad *configuration* takes down all replicas simultaneously (they're identical), so HA protects against *pod/node* failure but **not** against a *bad config* every replica shares.

Load-balancing insight (learned during testing): Kubernetes Service load balancing is connection-based (L4). A single browser reusing a keep-alive connection sticks to one pod, so per-pod metrics initially looked uneven. Testing through the ingress with fresh connections showed clean round-robin distribution across both pods. Lesson: *how you test determines what you observe* — `kubectl port-forward` pins to one pod and bypasses the load balancer.

7. Observability

Decision: kube-prometheus-stack (Prometheus + Grafana + Alertmanager) + loki-stack (Loki + Promtail), all via Helm.

Reasoning:

- Community Helm charts provide a working, production-grade stack quickly, letting effort focus on *using* observability (dashboards, alerts, incident response) rather than assembling it.
- The app exposes `/metrics`; a ServiceMonitor tells Prometheus to scrape it.
- Custom "Service Health" dashboard (8 panels) built by hand for service-level signals; the chart's built-in dashboards cover cluster/infrastructure metrics.
- Five alert rules, each mapped to a failure scenario. Recording rules pre-compute the error-ratio SLI and per-pod CPU.

Counter vs Gauge decision: `notes_created_total` is a Counter (per-pod, resets on restart) — good for *rates*, wrong for an absolute total across replicas. A DB-backed Gauge (`notes_in_db`, queried on each scrape) gives the *authoritative* total. Using the right metric type for the question is a deliberate, demonstrable choice.

k3s false-positive alerts (noted honestly): kube-prometheus-stack assumes standard Kubernetes topology. On k3s the control-plane components (controller-manager, scheduler, kube-proxy) run embedded in one process and don't expose the standard scrape endpoints, so the bundled `KubeControllerManagerDown` / `KubeProxyDown` / `KubeSchedulerDown` alerts fire as false positives. In production these would be silenced or reconfigured to match k3s.

8. Security Posture

Decision: Defense-in-depth appropriate to a single-node PoC (full detail in `docs/security.md`).

Controls implemented:

- Secrets in Kubernetes Secret + GitHub Secrets, never in Git (12-factor config).
- Non-root container on a minimal base image.
- Dedicated ServiceAccount with `automountServiceAccountToken: false` (app needs no k8s API access → least privilege).
- CI/CD deploys as a non-root user.
- Trivy image scanning in CI.
- NetworkPolicy declaring Postgres accepts traffic only from the app.

Honest limitation: default k3s uses Flannel, which does **not** enforce NetworkPolicies. The policy expresses correct *intent* but is not enforced; production would use a policy-capable CNI (e.g. Calico). Stating this shows understanding of the *declare-vs-enforce* distinction rather than overstating security.

Alerting notifications scoped out: the assignment requires alert *rules* (implemented, 5 of them). Notification *delivery* (email/Slack via Alertmanager) was deliberately deferred to protect time for incident-response documentation. Alertmanager is deployed and ready to be configured. Understanding the trade-off and scoping under deadline is itself an SRE skill.

9. Observed: Automated Exploit Scanning on Public Exposure

Within hours of exposing the service on a public Elastic IP, the `/metrics` endpoint recorded a stream of unsolicited 404 requests probing paths like `/vendor/phpunit/.../eval-stdin.php` and `/index.php` — automated bots scanning for known vulnerabilities. All were safely rejected (the app is Python/Flask, not PHP).

Why it matters: confirms that public exposure attracts automated attacks within minutes, and reinforces the value of a minimal attack surface and non-root containers. A spike in 404s from unknown paths is itself an alertable reconnaissance signal. Production follow-up: restrict the security group to known IPs, add a WAF / rate limiting.

10. Known Limitations (single-node PoC)

- **Single node** — the node is a single point of failure; no cross-node resilience.
- **In-cluster Postgres, single replica** — a single point of failure; production would use managed/replicated PostgreSQL with backups (PITR).
- **NetworkPolicy not enforced** (Flannel) — see §8.
- **Alert notifications not wired** (rules only) — see §8.
- These are acceptable for the evaluation scope and are called out honestly rather than hidden.